PI_Curs_10

10.1

Sunt încântat să vă ghidez prin lecția introductivă despre **Modele Generative**. Acest capitol marchează o tranziție de la sarcinile clasice de predicție la crearea de date noi, realiste.

Ghid de Studiu Complet: 10. Modele Generative

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 10 face trecerea de la învățarea discriminativă la **învățarea generativă** (sau modelarea generativă). În cea mai mare parte a cursului anterior, ne-am concentrat pe sarcini de predicție, în care rețelele neuronale profunde învățau **mapări de la date la etichete** (învățare discriminativă),,. **Învățarea generativă** depășește aceste sarcini, permițând modelului să **capteze concis caracteristicile distribuției datelor** dintr-un set mare, fără etichete,. Obiectivul final este de a **genera noi exemple de date sintetice** (imagini fotorealiste, de exemplu) care să semene plauzibil cu setul de date de antrenament,. Recenta dezvoltare a rețelelor neuronale profunde a deschis posibilitatea de a rezolva probleme de învățare generativă, o tendință majoră în ultimii ani,.

Care sunt cele 3 idei principale pe care trebuie neapărat să le rețin?

1. **Discriminativ vs. Generativ:** Învățarea discriminativă se concentrează pe **discriminarea (clasificarea)** între exemple (ex: pisici vs. câini),. Modelarea generativă se concentrează pe **captarea distribuției datelor** și **generarea de noi exemple sintetice**,.
2. **Succesul Discriminativ:** Rețelele neuronale antrenate prin *backpropagation* au obținut performanțe remarcabile în învățarea discriminativă pe seturi de date mari și complexe,.
3. **Noi Posibilități:** Succesul în învățarea discriminativă a deschis noi posibilități, inclusiv utilizarea rețelelor neuronale recurente (inițial antrenate pentru predicția caracterului următor) pentru a acționa ca **modele generative**,.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

10. Modele Generative – Introducere și Contrast

- **Cursul Anterior:** În mare parte a cursului, s-a discutat despre cum se fac **predicții**,.
- **Metoda:** S-au utilizat rețele neuronale profunde pentru a învăța **mapări de la exemple**

de date la etichete,.

- **Învățarea Discriminativă (*Discriminative Learning*):** Acest tip de învățare urmărește să **facă distincția (să discrimineze)** între fotografii (ex: fotografii cu pisici și fotografii cu câini),.
- **Exemple de Sarcini Discriminative:** Clasificatoarele și regressorii sunt ambele exemple de învățare discriminativă,.
- **Performanță:** Rețelele neuronale antrenate prin *backpropagation* au obținut performanțe remarcabile în învățarea discriminativă pe seturi de date complicate,. De exemplu, acuratețea clasificării pe imagini de înaltă rezoluție a trecut de la nivel inutil la nivel uman în doar 5-6 ani,.

Modelarea Generativă (*Generative Modeling*)

- Învățarea automată cuprinde mai mult decât doar rezolvarea sarcinilor discriminative,.
- **Obiectiv:** Dat fiind un set mare de date **fără etichete**, scopul este de a învăța un model care **captează concis caracteristicile distribuției datelor**,.
- **Generare:** După ce un astfel de model este obținut, se pot eșantiona **exemple de date sintetice** care să semene cu distribuția datelor de antrenament,.
- **Exemplu:** Dat fiind un corp mare de fotografii cu fețe, modelul ar trebui să poată **genera o nouă imagine fotorealistică** care să pară plauzibil că provine din același set de date,.

Tendințe și Posibilități Recente

- Până recent, nu exista o metodă capabilă să **sintetizeze imagini fotorealiste noi**,.
- **Succesul rețelelor neuronale profunde pentru învățarea discriminativă** a deschis noi posibilități în rezolvarea problemelor generative,.
- O tendință majoră recentă este aplicarea rețelelor discriminative profunde pentru a depăși provocările în probleme pe care nu le consideram în general ca fiind probleme de învățare supervizată,.
- **Exemplu de Conversie:** Modelele lingvistice recurente, antrenate inițial pentru a prezice caracterul următor (sarcină discriminativă), pot acționa, odată antrenate, ca un **model generativ**,.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Discriminative Learning (Învățarea Discriminativă)

- **Explicație Simplificată (Alegerea între A și B):** Învățarea discriminativă înseamnă a învăța **granița** sau **diferența** dintre clase. Dacă îți arăt o poză, modelul discriminativ îți

spune: "E 90% pisică și 10% câine". El nu știe *cum* arată o pisică sau un câine în detaliu, ci doar **ce face ca o pisică să nu fie un câine**.

Termen Tehnic: Generative Modeling (Modelarea Generativă)

- **Analogie (Artistul):** Un model generativ este ca un artist care învață stilul. Dacă îi arăți mii de fețe, el nu doar că le poate clasifica, dar învață **distribuția (forma, structura, culorile) fundamentală** a fețelor. Odată ce înțelege distribuția, poate **crea o față complet nouă** care nu a existat niciodată, dar care arată realist,.

Context Suplimentar: Succesul Discriminativ ca Premisă

- *Asta se întâmplă deoarece:* Deși sarcinile generative păreau de nerezolvat, succesul remarcabil al rețelelor neuronale (CNN-uri, în special) în învățarea discriminativă a demonstrat că aceste rețele pot învăța reprezentări interne extrem de bogate ale datelor. Această capacitate de a procesa și codifica informațiile complexe este exact baza necesară pentru a putea, ulterior, să **genereze** date noi,.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Schimbarea de Paradigmă):**
  - **CAUZĂ:** Succesul rețelelor neuronale profunde în învățarea discriminativă (rezolvarea sarcinilor clasice de predicție),,,.
  - **EFECT:** Rețelele au demonstrat o capacitate suficientă de a învăța reprezentări complexe, ceea ce a deschis posibilitatea de a aborda **învățarea generativă** (crearea de imagini fotorealiste noi),,,.
- **Conexiunea cu Secțiunile Anterioare:** Toate sarcinile studiate anterior (clasificarea, regresia, *image captioning*, VQA, etc.) erau, în esență, **sarcini discriminatorii** (învățarea unei mapări de la intrare la o etichetă/ieșire specifică),. Modelarea generativă reprezintă pasul următor, nesupervizat sau slab supervizat.

---

5. EXERCITII (Analiză și Comparație)

Exercițiul G.1: Analiza Comparativă a Învățării

Completați următorul tabel comparativ, bazat pe informațiile din surse:

Criteriu	Învățare Discriminativă	Modelare Generativă
Obiectiv principal	Să facă distincția (să discrimineze) între exemple.	Să capteze concis caracteristicile **distribuției datelor**.

Criteriu	Învățare Discriminativă	Modelare Generativă
Tip de ieșire	O predicție (etichetă sau valoare numerică).	Exemple de date **sintetice** care seamănă cu datele de antrenament.
Exemplu de sarcină	Clasificarea fotografiilor (pisici vs. câini).	Generarea unei noi imagini **fotorealiste** care să arate plauzibil.

Rezolvare G.1:

Criteriu	Învățare Discriminativă	Modelare Generativă
Obiectiv principal	Să facă distincția (să discrimineze) între exemple,.	Să capteze concis caracteristicile **distribuției datelor**,.
Tip de ieșire	O predicție (etichetă sau valoare numerică).	Exemple de date **sintetice** care seamănă cu distribuția datelor de antrenament,.
Exemplu de sarcină	Clasificarea fotografiilor (pisici vs. câini),.	Generarea unei noi imagini **fotorealiste** care să arate plauzibil (ex: fețe),.

Exercițiul G.2: Analiza Conversiei de Sarcini

1. Explicați cum un **model lingvistic recurent** (RNN language model) poate fi considerat un exemplu de utilizare a unei rețele discriminatorii pentru a acționa ca un model generativ,.
2. De ce, până recent, nu exista o metodă eficientă pentru a sintetiza imagini fotorealiste noi,?

Rezolvare G.2:

1. **Conversia Modelului Lingvistic:** Un model lingvistic recurent este antrenat inițial pentru a prezice **caracterul următor** (ceea ce este o sarcină discriminatorie de clasificare),. Totuși, odată antrenat, modelul poate fi folosit pentru a genera secvențe complete de text (caracter după caracter), acționând astfel ca un **model generativ**,.
2. **Lipsa Metodelor:** Până recent, nu exista o metodă care să poată sintetiza imagini fotorealiste noi. Succesul rețelelor neuronale profunde în învățarea discriminativă a deschis noi posibilități în acest domeniu,.

10.2

Sunt onorat să vă fiu profesor și tutore în explorarea Rețelelor Generative Adversariale (GANs), una dintre cele mai importante inovații din inteligența artificială recentă.

Ghid de Studiu Complet: 10.1 Generative Adversarial Networks (GANs)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 10.1 acoperă conceptul fundamental al **Rețelelor Generative Adversariale (GANs)**, introduse printr-o lucrare revoluționară în **2014**,. GANs reprezintă o metodă inteligentă de a utiliza puterea modelelor discriminative pentru a obține modele generative performante,. La bază, ideea este că un generator de date este considerat bun dacă nu putem distinge datele false de datele reale,. Acest principiu este formalizat ca un **joc "minimax"** în care două rețele, Generatorul (G) și Discriminatorul (D), se află în competiție,,,. Discriminatorul învață să clasifice corect dacă datele sunt reale sau false, iar Generatorul învață să producă date suficient de realiste încât să păcălească Discriminatorul, chiar dacă acesta este o rețea neuronală profundă de ultimă generație,.

Care sunt cele 3 idei principale pe care trebuie neapărat să le rețin?

1. **Arhitectura Bicomponentă:** GANs sunt formate din două rețele aflate în competiție: **Generatorul (G)** și **Discriminatorul (D)**,,.
2. **Principiul Testului cu Două Eșantioane:** Succesul Generatorului este definit de incapacitatea unui clasificator (Discriminatorul) de a distinge setul de date generate (X') de cel real (X) (similar cu *two-sample test* din statistică),.
3. **Jocul Minimax:** Antrenamentul GANs implică optimizarea unei funcții obiectiv complexe în care D încearcă să maximizeze funcția, iar G încearcă să o minimizeze (sau invers, în funcție de formulare),.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

10.1 Generative Adversarial Networks (GANs)

- **Introducere:** Conceptul GANs a fost introdus în **2014**,.
- **Principiul de Bază:** Un generator de date este considerat bun dacă nu se poate spune dacă datele false sunt diferite de datele reale,.
- **Analogie Statistică:** În statistică, acest lucru este numit **"two-sample test"** (testul cu două eșantioane), care răspunde la întrebarea dacă seturile de date X și X' au fost extrase din aceeași distribuție,.
- **Utilizarea Constructivă:** GANs folosesc acest test cu două eșantioane într-un mod constructiv, furnizând **semnale de antrenament** către modelul generativ pentru a-l îmbunătăți,.
- **Scopul Generatorului:** Generatorul trebuie să păcălească clasificatorul (Discriminatorul), chiar dacă clasificatorul este o rețea neuronală profundă de ultimă generație,.

Arhitectura GANs (Generator și Discriminator)

Arhitectura GAN are două componente principale,,,:

1. **Rețeaua Generator (G):**
   - Este un dispozitiv (de exemplu, o rețea profundă sau chiar un motor de randare) capabil să genereze date false ($x' = G(z)$) care să arate ca datele reale,,,.
   - **Intrare:** Generatorul desenează un parametru $\mathbf{z} \in \mathbb{R}^d$ dintr-o sursă de aleatoriu (ex: o distribuție normală $\mathbf{z} \sim \mathcal{N}(0, 1)$),. Acest $\mathbf{z}$ este adesea numit **variabila latentă**,.
   - **Obiectiv:** Să păcălească Discriminatorul să clasifice datele generate $\mathbf{x}' = G(\mathbf{z})$ ca fiind date reale, adică $D(G(\mathbf{z})) \approx 1$,.
2. **Rețeaua Discriminator (D):**
   - Este un **clasificator binar** care încearcă să distingă datele false de datele reale,,,.
   - **Ieșire:** Discriminatorul produce o predicție scalară $o \in \mathbb{R}$,. Acesta folosește o funcție sigmoid pe un strat complet conectat cu mărimea ascunsă 1 pentru a obține probabilitatea prezisă $D(\mathbf{x}; \theta_d) = 1/(1 + e^{-o})$,.
   - **Etichete presupuse:** Eticheta y pentru datele reale este 1, iar pentru datele false este 0,.

Antrenamentul Adversarial și Funcția Obiectiv (Minimax Game)

Cele două rețele sunt în competiție,: Generatorul încearcă să păcălească Discriminatorul, iar Discriminatorul se adaptează la datele false noi, iar această informație este folosită, la rândul ei, pentru a îmbunătăți Generatorul,.

- **Obiectivul Discriminatorului (D):** Minimizează pierderea *cross-entropy* (să clasifice corect x ca fiind real, și $G(z)$ ca fiind fals),.

$$\min_D -y \log D(\mathbf{x}) - (1 - y) \log(1 - D(\mathbf{x}))$$

- **Obiectivul Generatorului (G):** Maximizează pierderea *cross-entropy* când $y = 0$ (pentru a forța $D(G(\mathbf{z}))$ să fie aproape de 1),.

$$\max_G -(1 - y) \log(1 - D(G(\mathbf{z}))) = \max_G -\log(1 - D(G(\mathbf{z})))$$

- **Forma Obiectivului (Comună):** În practică, pentru a evita gradientul prea mic atunci când Generatorul se descurcă prost, se **minimizează** următoarea pierdere pentru G (echivalent cu a da eticheta $y = 1$ la $G(z)$),:

$$\min_G -\log(D(G(\mathbf{z})))$$

- **Funcția Obiectiv Comprehensivă (Minimax Game):** D și G joacă un joc "minimax",:

$$\min_G \max_D {}_{\mathbf{x} \sim Data}[\log D(\mathbf{x})] + {}_{\mathbf{z} \sim Noise}[\log(1 - D(G(\mathbf{z})))]$$

Procedura de Antrenament (Stochastic Gradient Descent)

Antrenarea GANs folosește *stochastic gradient descent*,,,:

1. Pentru un număr de iterații de antrenament:
2. Pentru k **pași**, se actualizează Discriminatorul D (cu parametri θ_d):
   - Se eșantionează mini-batch-uri de zgomot $\mathbf{z}^{(i)}$ și exemple reale $\mathbf{x}^{(i)}$,.
   - Se folosește *stochastic gradient descent* pentru a actualiza θ_d,.
3. Se actualizează Generatorul G (cu parametri θ_g):
   - Se eșantionează mini-batch-uri de zgomot $\mathbf{z}^{(i)}$,.
   - Se folosește *stochastic gradient ascent* pe pierderea originală (sau *stochastic gradient descent* pe pierderea comună minimizată),.
4. În experimente, hiperparametrul k este adesea folosit cu valoarea $k = 1$,.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Two-Sample Test (Testul cu Două Eșantioane)

- **Explicație Simplificată (Autenticitatea Banii):** În statistică, un *two-sample test* te întreabă: "Ai două grămezi de bani (unul real, unul fals). Au fost făcute amândouă în aceeași monetărie?". GANs folosesc acest principiu. Discriminatorul este judecătorul care încearcă să spună "nu, cele două grămezi nu provin din aceeași distribuție",. Prin feedback-ul Discriminatorului, Generatorul învață să-și îmbunătățească "monetăria" până când judecătorul nu mai poate face diferența,.

Concept Dificil: Variabila Latentă ($\mathbf{z}$)

- **Analogia (Sămânța Creativității):** Variabila latentă $\mathbf{z}$ (extrasă dintr-o distribuție normală, ex: $\mathcal{N}(0, 1)$) este ca o **"sămânță" de zgomot** complet aleatorie,. Generatorul (G) ia această sămânță și o transformă într-un exemplu complex și coerent de date (o imagine, un sunet etc.),. Fiecare sămânță z diferită ar trebui să genereze o ieșire unică, dar plauzibilă.

Context Suplimentar: Maximizarea vs. Minimizarea Pierderii Generatorului

- *Asta se întâmplă deoarece:* Deși obiectivul teoretic al Generatorului este să maximizeze $\log D(G(\mathbf{z}))$ (sau să minimizeze $-\log D(G(\mathbf{z}))$), formula originală de maximizare a Discriminatorului ($\max_G -\log(1 - D(G(\mathbf{z})))$) are o problemă. La începutul antrenamentului, când Generatorul este slab, $D(G(\mathbf{z})) \approx 0$, iar gradientul funcției este foarte mic (aproape de zero). Prin urmare, se folosește formularea echivalentă de minimizare $\min_G -\log(D(G(\mathbf{z})))$,, care oferă un gradient mai puternic pentru îmbunătățirea inițială.

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Învățarea Discriminativă:** GANs sunt o metodă inteligentă de a obține modele generative, însă ele se bazează pe puterea **modelelor discriminative** (rețeaua Discriminator) pentru a învăța,. Succesul GANs demonstrează că se poate folosi învățarea discriminativă în mod constructiv pentru a antrena Generatorul,.
- **Implicația Competiției:** Competiția adversarială forțează Generatorul să producă date din ce în ce mai realiste, deoarece Discriminatorul se adaptează continuu la datele false noi,. Acest mecanism de îmbunătățire reciprocă este motivul pentru care GANs au obținut rezultate spectaculoase în sinteza de imagini,.

---

5. EXERCITII (Analiză și Formulă)

Exercițiul GAN.1: Analiza Funcției Obiectiv (Minimax Game)

Considerați funcția obiectiv comprehensivă GAN:

$$\min_G \max_D {}_{\mathbf{x}\sim Data}[\log D(\mathbf{x})] + {}_{\mathbf{z}\sim Noise}[\log(1 - D(G(\mathbf{z})))]$$

1. Explicați acțiunile pe care le întreprinde Discriminatorul D pentru a maximiza obiectivul.
2. Explicați ce se întâmplă cu termenul ${}_{\mathbf{x}\sim Data}[\log D(\mathbf{x})]$ și cu termenul ${}_{\mathbf{z}\sim Noise}[\log(1 - D(G(\mathbf{z})))]$ dacă Discriminatorul D reușește perfect să își îndeplinească obiectivul.

Rezolvare GAN.1:

1. **Acțiunile Discriminatorului (D):** Discriminatorul încearcă să maximizeze funcția. Pentru a face acest lucru, el trebuie să atribuie o probabilitate $D(\mathbf{x})$ cât mai mare datelor reale $\mathbf{x}$ (făcând $\log D(\mathbf{x}) \approx 0$) și să atribuie o probabilitate $D(G(\mathbf{z}))$ cât mai mică datelor false $G(\mathbf{z})$ (făcând $\log(1 - D(G(\mathbf{z})))$ cât mai mare, sau $D(G(\mathbf{z})) \approx 0$),.
2. **Când D reușește perfect:**
   - $D(\mathbf{x}) \approx 1$. Prin urmare, $\log D(\mathbf{x}) \approx \log(1) = 0$. Primul termen se apropie de **zero**.
   - $D(G(\mathbf{z})) \approx 0$. Prin urmare, $\log(1 - D(G(\mathbf{z}))) \approx \log(1 - 0) = 0$. Al doilea termen se apropie și el de **zero**.
   - Valoarea maximă (cea mai puțin negativă) a funcției obiectiv este atinsă când ambii termeni se apropie de zero (presupunând logaritm natural sau baza e).

Exercițiul GAN.2: Analiza Procedurii de Antrenament (k și m)

Sistemul de antrenament GAN utilizează *stochastic gradient descent* și hiperparametrul k.

1. Explicați rolul hiperparametrului k în antrenamentul GAN și care este valoarea sa

utilizată frecvent în experimente,.

2. În formula de actualizare a Discriminatorului (linia 5 din sursă,), ce reprezintă m și care este rolul sumei $\frac{1}{m}\sum_{i=1}^{m}(\ldots)$?

Rezolvare GAN.2:

1. **Rolul Hiperparametrului k:** k reprezintă numărul de pași sau iterații prin care este actualizat **Discriminatorul (D)** pentru **fiecare pas** de actualizare a Generatorului (G),. Valoarea utilizată frecvent în experimente este $k = 1$,.
2. **Rolul lui m în Formula D:** m reprezintă **mărimea mini-batch-ului**,. Termenul $\frac{1}{m}\sum_{i=1}^{m}(\ldots)$ reprezintă **media pierderii *cross-entropy*** calculată pe mini-batch-ul de m exemple (reale $x^{(i)}$ și generate $G(z^{(i)})$),.

10.3

Sunt la dispoziția dumneavoastră pentru a genera un Ghid de Studiu complet pentru lecția **10.2 DCGANs (Deep Convolutional Generative Adversarial Networks)**, care detaliază cum au fost stabilizate Rețelele Generative Adversariale pentru utilizarea cu Rețele Convoluționale Profunde.

Ghid de Studiu Complet: 10.2 DCGANs (Deep Convolutional Generative Adversarial Networks)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 10.2 DCGANs abordează problema instabilității antrenării Rețelelor Generative Adversariale (GANs) atunci când sunt folosite Rețele Convoluționale (CNNs) pentru modelarea imaginilor, întrucât încercările anterioare de a scala GANs folosind CNN-uri fuseseră în mare parte nereușite. Prin explorări extinse de modele, a fost identificată o **familie de arhitecturi** care a permis o antrenare stabilă pe o gamă largă de seturi de date, ducând la posibilitatea antrenării de modele generative **mai profunde și cu rezoluție mai înaltă**. Baza acestei abordări constă în adoptarea și modificarea a trei schimbări demonstrate anterior la arhitecturile CNN. Aceste schimbări se axează pe eliminarea straturilor deterministe de *pooling*, eliminarea straturilor complet conectate (FC) ascunse și utilizarea **Batch Normalization (BN)** pentru stabilizare.

Care sunt cele 3 idei principale pe care trebuie neapărat să le rețin?

1. **Stabilitate prin Arhitectură:** DCGANs au rezolvat problema antrenării instabile a GANs cu CNN-uri, permițând antrenarea de modele generative **mai profunde și cu rezoluție mai înaltă**.
2. **Înlocuirea Pooling-ului și Straturilor FC:** Se înlocuiesc funcțiile de *pooling* spațial determinist (cum ar fi *max-pooling*) cu **convoluții cu *stride*** (pentru *downsampling* în

Discriminator) și se elimină straturile complet conectate ascunse.
3. **Rolul Batch Normalization (BN):** BN este critică pentru a **stabiliza antrenamentul**, ajută la rezolvarea problemelor ce decurg din inițializarea slabă și previne **colapsul generatorului la un singur punct** (*collapsing all samples to a single point*).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

10.2 DCGANs (Deep Convolutional Generative Adversarial Networks)

- Încercările anterioare de a scala GANs folosind CNN-uri pentru a modela imagini au fost nereușite.
- DCGANs a identificat o familie de arhitecturi care a permis **antrenarea stabilă** pe diverse seturi de date, permițând antrenarea de modele generative mai profunde și cu rezoluție mai înaltă.
- Miezul abordării DCGAN constă în adoptarea și modificarea a trei schimbări demonstrate anterior la arhitecturile CNN.

Cele Trei Schimbări Arhitecturale Cruciale

1. **Rețeaua Complet Convoluțională și Eliminarea Pooling-ului:**
   - Se înlocuiesc funcțiile deterministe de *pooling* spațial (cum ar fi *max-pooling*) cu **convoluții cu *stride***.
   - Această abordare este utilizată în Discriminator pentru a învăța propria subsamplare spațială (*downsampling*).
   - Este utilizată și în Generator, permițându-i să învețe propria supraeșantionare spațială (*upsampling*) prin **convoluții transpuse**.
2. **Eliminarea Straturilor Complet Conectate (FC) Ascunse:**
   - Există o tendință generală de a elimina straturile complet conectate deasupra *feature*-urilor convoluționale.
   - **Global Average Pooling (GAP)**, utilizat în modelele de clasificare a imaginilor, s-a dovedit că mărește stabilitatea modelului, dar încetinește viteza de convergență.
   - O cale de mijloc, care a funcționat bine, a fost conectarea directă a celor mai înalte *feature*-uri convoluționale la intrarea și ieșirea modelului (pentru Generator, respectiv Discriminator).
   - În Discriminator, ultimul strat convoluțional este **aplatizat (*flattened*)** și apoi introdus într-o singură ieșire sigmoidă.
   - Primul strat al GAN, care preia distribuția uniformă de zgomot $\mathbf{z}$ ca intrare, ar putea fi numit complet conectat, dar rezultatul este **remodelat (*reshaped*) într-un tensor 4D** și utilizat ca început al stivei convoluționale.
3. **Utilizarea Batch Normalization (BN):**

- BN **normalizează intrarea** către fiecare unitate pentru a avea medie zero și varianță unitară, ceea ce **stabilizează învățarea**.
- BN ajută la tratarea problemelor de antrenament care apar din **inițializarea slabă** și ajută la fluxul de gradient în modelele mai profunde.
- BN s-a dovedit critică pentru a permite Generatorilor profunzi să înceapă să învețe, prevenind o defecțiune comună în GANs: **colapsarea tuturor exemplelor într-un singur punct**.
- S-a evitat aplicarea directă a BN pe stratul de ieșire al Generatorului și pe stratul de intrare al Discriminatorului, deoarece acest lucru a dus la **oscilația eșantionului și instabilitatea modelului**.

Funcțiile de Activare Recomandate

- **Generator:** Se folosește activarea **ReLU** (Rectified Linear Unit) pentru toate straturile, cu excepția stratului de ieșire, care folosește funcția **tanh**.
  - S-a observat că utilizarea unei activări delimitate (*bounded activation*) precum tanh pe ieșire permite modelului să satureze și să acopere mai rapid **spațiul de culoare** al distribuției de antrenament.
- **Discriminator:** Se folosește activarea **Leaky ReLU** pentru toate straturile.
  - S-a constatat că Leaky ReLU funcționează bine, în special pentru **modelarea cu rezoluție mai înaltă**.
  - Acest lucru contrastează cu lucrarea GAN originală, care utiliza activarea *maxout*.

Exemplu Arhitectural (Generator DCGAN)

- Figura 2 ilustrează un generator DCGAN utilizat pentru modelarea scenei LSUN (pe setul de date cu dormitoare).
- Intrarea este o distribuție uniformă $\mathbf{z}$ de **100 de dimensiuni**.
- $\mathbf{z}$ este proiectată și remodelată (*reshape*) într-o reprezentare convoluțională spațială mică, cu multe hărți de *feature* (ex: dimensiune 4×4 cu 1024 de hărți de *feature* după primul pas).
- O serie de **patru convoluții transpuse** (CONV 1, 2, 3, 4) convertesc această reprezentare de nivel înalt într-o imagine de 64×64 **pixeli**.
- **Observație Notabilă:** Nu sunt utilizate straturi complet conectate sau straturi de *pooling*.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Convoluții Transpuse (Transposed Convolutions)

- **Explicație Simplificată (Desenul la Generator):** Convoluțiile transpuse (adesea numite și *deconvoluții*) sunt folosite în Generator pentru **upsampling** (creșterea rezoluției). Dacă o convoluție normală ia o imagine mare și o face mai mică, o convoluție transpusă ia o hartă de *feature* mică (ex: 8×8) și, folosind filtre învățate, o transformă într-o hartă mai mare (ex: 16×16), adăugând detalii.

Concept Dificil: Colapsarea Generatorului (Mode Failure)

- **Analogia (Artistul Plictisit):** O defecțiune comună a GANs (evitată prin BN) este *mode collapse* sau colapsarea Generatorului la un singur punct. Imaginați-vă că Generatorul trebuie să deseneze fețe diverse, dar descoperă că o singură față simplă păcălește Discriminatorul suficient de bine. Atunci, Generatorul începe să genereze **aceeași singură față** la nesfârșit, indiferent de variabila latentă $\mathbf{z}$ de intrare. **Batch Normalization** a fost critică pentru a preveni acest colaps, permițând Generatorului să exploreze distribuția datelor.

Context Suplimentar: Leaky ReLU vs. ReLU vs. tanh

- *Asta se întâmplă deoarece:* **Leaky ReLU** este preferat în Discriminator deoarece introduce o mică scurgere (*leak*) pentru intrările negative. Acest lucru ajută Discriminatorul să aibă un gradient mai bun, chiar și pentru intrările false, îmbunătățind modelarea cu rezoluție mai înaltă. **tanh** este folosit pe ieșirea Generatorului deoarece este o funcție delimitată (între -1 și 1), ceea ce ajută modelul să acopere rapid **spațiul de culoare** al imaginii de antrenament.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Eliminarea Pooling-ului):**
  - **CAUZĂ:** Dorința de a permite rețelei să învețe propria subsamplare/supraeșantionare spațială, în loc să folosească funcții deterministe (Max-Pooling).
  - **EFECT:** Înlocuirea *pooling*-ului cu **convoluții cu *stride*** (pentru Discriminator) și **convoluții transpuse** (pentru Generator).
- **Conexiunea cu Secțiunea 10.1 (GANs):** DCGANs este o specializare a GANs care aplică principii robuste de arhitectură CNN pentru a rezolva problema majoră a **instabilității antrenamentului** (rezultat din eșecurile anterioare de a scala GANs folosind CNNs).

---

5. EXERCITII (Calcul și Analiză Arhitecturală)

Exercițiul D.1: Analiza Arhitecturii Generatorului DCGAN

Un Generator DCGAN folosește o distribuție uniformă de zgomot z **de 100 de dimensiuni** ca intrare, care este proiectată și remodelată la începutul stivei convoluționale. Generatorul produce o imagine finală de 64×64 **pixeli**.

1. Dacă generatorul produce imagini RGB, câte canale are tensorul de ieșire finală G(z)?
2. Explicați de ce primul strat care primește inputul $\mathbf{z}$ este totuși considerat ca fiind conectat direct la *feature*-urile convoluționale, deși ar putea fi numit complet conectat.

Rezolvare D.1:

1. **Numărul de Canale de Ieșire:** O imagine RGB are **3 canale** (Roșu, Verde, Albastru). Tensorul final de ieșire este, prin urmare, $64 \times 64 \times 3$ (dimensiunea imagine).
2. **Conexiunea cu Feature-urile Convoluționale:** Primul strat care primește $\mathbf{z}$ este *numit* complet conectat (într-un sens). Totuși, rezultatul său este **remodelat într-un tensor 4D** (spațiu, spațiu, adâncime, batch) și este folosit **direct ca începutul stivei convoluționale**, fără straturi FC ascunse ulterioare.

Exercițiul D.2: Analiza Batch Normalization (BN)

1. Ce problemă de stabilitate rezolvă BN, care apare frecvent în Generatorii profunzi din arhitecturile GAN anterioare?
2. De ce, în arhitectura DCGAN, se evită aplicarea Batch Normalization la stratul de ieșire al Generatorului și la stratul de intrare al Discriminatorului?

Rezolvare D.2:

1. **Problema Stabilității Rezolvată:** BN ajută la rezolvarea problemelor de antrenament care apar din inițializarea slabă și este critică pentru a preveni **colapsarea Generatorului la un singur punct** (*collapsing all samples to a single point*), care este o defecțiune comună observată în GANs.
2. **Excluderea BN:** Aplicarea directă a BN la stratul de ieșire al Generatorului și la stratul de intrare al Discriminatorului a rezultat în **oscilația eșantionului și instabilitatea modelului**.

Exercițiul D.3: Analiza Funcțiilor de Activare

Completați tabelul cu funcțiile de activare recomandate pentru DCGANs:

Componentă	Strat	Funcție de Activare Recomandată
Generator	Toate straturile ascunse	
Generator	Stratul de Ieșire	
Discriminator	Toate straturile	

Rezolvare D.3:

Componentă	Strat	Funcție de Activare Recomandată
Generator	Toate straturile ascunse	**ReLU**
Generator	Stratul de Ieșire	**tanh**
Discriminator	Toate straturile	**Leaky ReLU**

10.4

Iată o serie de întrebări și probleme de calcul detaliate pe baza surselor furnizate.

EXAMEN SIMULAT: MODELE GENERATIVE ȘI ARHITECTURI GENERATIVE ADVERSARIALE (GANs & DCGANs)

I. Întrebări de Analiză și Sinteză (Tip "Bacalaureat")

Subiectul I.1: Fundamentele Modelării Generative

Sarcină: Definiți **Modelarea Generativă** (Generative Modeling) și explicați cum diferă esențial de **Învățarea Discriminativă** (Discriminative Learning), pe care s-a pus accentul în majoritatea părților anterioare ale cursului. Oferiți un exemplu de sarcină pentru fiecare tip de învățare.

Rezolvare I.1 (Sinteză):

Învățarea Discriminativă se concentrează pe **predicție** și învățarea mapărilor de la exemple de date la etichete,. Obiectivul său este să facă distincția (*să discrimineze*) între exemple, (de exemplu, clasificatori și regressorii,). Un exemplu este **clasificarea** fotografiilor în *pisici* sau *câini*,.

Modelarea Generativă depășește aceste sarcini,. Dat fiind un set mare de date **fără etichete**, scopul este de a învăța un model care **captează concis caracteristicile distribuției datelor**,. Odată ce un astfel de model este obținut, se pot eșantiona **exemple de date sintetice** care seamănă cu distribuția datelor de antrenament,. Un exemplu este generarea unei **noi imagini fotorealiste** care să arate plauzibil, dintr-un corp mare de fotografii cu fețe,.

Subiectul I.2: Principiul Antrenamentului Adversarial (GANs)

Sarcină: Explicați principiul pe care se bazează Rețelele Generative Adversariale (GANs), inclusiv cum folosesc un concept statistic într-un mod constructiv. Detaliați rolul fiecăreia dintre cele două rețele în **competiția** lor (jocul minimax) și care este obiectivul final al Generatorului (G).

Rezolvare I.2 (Analiză Principială):

Principiul de bază al GANs, introduse în 2014,, este că un generator de date este considerat bun dacă nu se poate spune dacă datele false sunt diferite de cele reale,.

Acest principiu este similar cu **testul cu două eșantioane (*two-sample test*)** din statistică, care răspunde la întrebarea dacă seturile de date X (real) și X' (fals) au fost extrase din aceeași distribuție,. GANs folosesc acest test statistic într-un **mod constructiv**, oferind semnale de antrenament Generatorului,.

Rolurile în Competiție (Minimax Game):

1. **Discriminatorul (D):** Este un clasificator binar care încearcă să **distingă datele false de datele reale**,,. Discriminatorul încearcă să **maximizeze** funcția obiectiv, atribuind probabilitate mare datelor reale ($\mathbf{x}$) și probabilitate mică datelor generate ($G(\mathbf{z})$),.
2. **Generatorul (G):** Ia un parametru $\mathbf{z}$ (variabila latentă) dintr-o sursă de aleatoriu și generează date false $\mathbf{x}' = G(\mathbf{z})$,. Obiectivul său este să **păcălească Discriminatorul** să clasifice $\mathbf{x}'$ ca fiind dat real,,, adică $D(G(\mathbf{z})) \approx 1$. Generatorul încearcă să **minimizeze** funcția obiectiv.

Generatorul trebuie să păcălească Discriminatorul, chiar dacă acesta este o rețea neuronală profundă de ultimă generație,.

Subiectul I.3: Stabilitatea DCGANs prin Modificări CNN

Sarcină: Explicați de ce încercările anterioare de a scala GANs folosind CNNs au fost nereușite și cum a rezolvat DCGAN această instabilitate prin implementarea celor **trei schimbări arhitecturale** majore. Detaliați rolul specific al **Batch Normalization (BN)** în Generator.

Rezolvare I.3 (Analiză Arhitecturală DCGAN):

Încercările anterioare de a scala GANs folosind CNN-uri pentru modelarea imaginilor au fost în mare măsură nereușite, deoarece antrenarea devenea instabilă,. DCGAN a rezolvat acest lucru identificând o familie de arhitecturi care permite **antrenarea stabilă** și antrenarea de modele generative mai profunde și cu rezoluție mai înaltă,.

Cele trei schimbări cruciale au fost:

1. **Înlocuirea Pooling-ului:** Funcțiile deterministe de *pooling* spațial (ex: *max-pooling*) sunt înlocuite cu **convoluții cu *stride*** (în Discriminator pentru *downsampling*) și **convoluții transpuse** (în Generator pentru *upsampling*), permițând rețelei să învețe subsamplarea/supraeșantionarea,,.
2. **Eliminarea Straturilor FC Ascunse:** Straturile complet conectate ascunse au fost eliminate. Între *feature*-urile convoluționale superioare și intrarea/ieșirea modelului, s-a adoptat o cale de mijloc: în Discriminator, ultimul strat convoluțional este aplatizat și introdus într-o singură ieșire sigmoidă,.

3. **Utilizarea Batch Normalization (BN):** BN normalizează intrarea către fiecare unitate, ceea ce **stabilizează învățarea**,. BN a fost critică pentru a permite Generatorilor profunzi să înceapă să învețe, prevenind o defecțiune comună a GANs: **colapsarea tuturor exemplelor într-un singur punct**,. Totuși, BN este evitată la stratul de ieșire al Generatorului și la stratul de intrare al Discriminatorului, deoarece aplicarea directă a dus la instabilitate,,.

II. Exerciții de Calcul și Analiză Detaliată

Exercițiul C.4: Analiza Funcției Obiectiv GAN (Pierderea Discriminatorului)

Se dă formula de minimizare a pierderii *cross-entropy* pentru Discriminator D:

$$\min_D -y \log D(\mathbf{x}) - (1-y)\log(1-D(\mathbf{x}))$$

,

Presupunând că $\mathbf{x}$ reprezintă date reale (eticheta $y=1$) și $G(\mathbf{z})$ reprezintă date false (eticheta $y=0$):

1. Determinați pierderea Discriminatorului D pentru un exemplu de date **reale** $\mathbf{x}$ (înlocuind $y=1$ în formulă).
2. Determinați pierderea Discriminatorului D pentru un exemplu de date **false** $G(\mathbf{z})$ (înlocuind $y=0$ și $\mathbf{x}=G(\mathbf{z})$ în formulă).

Rezolvare C.4:

1. **Pierderea pentru date reale ($\mathbf{x}$, $y=1$):**

$$\text{Loss}_D(\mathbf{x}) = -(1)\log D(\mathbf{x}) - (1-1)\log(1-D(\mathbf{x}))$$
$$\text{Loss}_D(\mathbf{x}) = -\log \mathbf{D(x)}$$

2. **Pierderea pentru date false ($G(\mathbf{z})$, $y=0$):**

$$\text{Loss}_D(G(\mathbf{z})) = -(0)\log D(G(\mathbf{z})) - (1-0)\log(1-D(G(\mathbf{z})))$$
$$\text{Loss}_D(G(\mathbf{z})) = -\log\mathbf{(1-D(G(z)))}$$

Exercițiul C.5: Analiza Arhitecturii Generatorului DCGAN (Convoluții Transpuse)

Conform Figurii 2, Generatorul DCGAN transformă o intrare $100z$ într-o imagine de 64×64 pixeli.

1. Dacă primul strat (după proiectare și remodelare) începe cu o reprezentare de 4×4 cu 1024 hărți de *feature*, câte convoluții transpuse cu *stride* 2 sunt necesare pentru a ajunge la rezoluția finală de 64×64? (Presupuneți că fiecare convoluție transpusa cu *stride* 2 dublează dimensiunea spațială, ignorând *padding*-ul).
2. Ce funcție de activare este utilizată la **stratului de ieșire** $G(\mathbf{z})$ și de ce?

Rezolvare C.5:

1. **Numărul de Convoluții Transpuse:** Rezoluția de pornire este 4.
   - Pasul 1 (CONV 1): $4 \times 2 = 8$
   - Pasul 2 (CONV 2): $8 \times 2 = 16$
   - Pasul 3 (CONV 3): $16 \times 2 = 32$
   - Pasul 4 (CONV 4): $32 \times 2 = 64$ Sunt necesare $\mathbf{4}$ convoluții transpuse pentru a ajunge la 64×64,.
2. **Funcția de Activare de Ieșire:** Stratul de ieșire folosește funcția **tanh**,. Aceasta este utilizată deoarece este o funcție delimitată (*bounded activation*) care permite modelului să satureze și să acopere mai rapid **spațiul de culoare** al distribuției de antrenament,.

Exercițiul C.6: Protocolul de Antrenament GAN

Protocolul de antrenament GAN utilizează *stochastic gradient descent* și hiperparametrul k, utilizat tipic cu valoarea $k = 1$.

1. Descrieți ordinea de actualizare a Discriminatorului (D) și a Generatorului (G) într-o iterație de antrenament GAN.
2. De ce, în practica comună de antrenament, se minimizează pierderea Generatorului folosind $\min_G -\log(D(G(\mathbf{z})))$, în loc să se maximizeze $\max_G \log(1 - D(G(\mathbf{z})))$?,

Rezolvare C.6:

1. **Ordinea de Actualizare:** Antrenamentul GAN implică o buclă for:
   - D **Actualizare:** Se execută k pași de actualizare a Discriminatorului D (cu parametri θ_d) folosind *stochastic gradient descent*,.
   - G **Actualizare:** Se execută 1 pas de actualizare a Generatorului G (cu parametri θ_g),.
2. **Motivația Schimbării Pierderii Generatorului:** Formula originală de maximizare a Generatorului ($\max_G \log(1 - D(G(\mathbf{z})))$) are o problemă la începutul antrenamentului. Atunci când Generatorul este slab, $D(G(\mathbf{z})) \approx 0$, iar gradientul pentru această pierdere devine **prea mic** pentru a face progrese bune,. Prin urmare, se folosește formularea de minimizare $\min_G -\log(D(G(\mathbf{z})))$,, care oferă un **gradient mai puternic** pentru a forța Generatorul să se îmbunătățească.